APEX

Autonomous Portfolio Execution Agent

FINAL BIBLE — HashKey Chain Horizon Hackathon | AI Track | Apr 2026

AI agents that autonomously manage on-chain RWA portfolios.

HSP payment settlement. NexaID identity. Chainlink price feeds.

Prize target: $5K (AI 1st) + Atlas Grant + HashKey Capital pipeline

github.com/henrysammarfo | HashKey Chain Testnet (Chain ID: 133)

PART 1 — ECOSYSTEM INTELLIGENCE

Full research done before touching an idea. Here is everything that matters about HashKey Chain and this hackathon.

What HashKey Chain Is

- Ethereum L2 built on OP Stack. EVM compatible — same Solidity, Hardhat, and ethers.js tooling.
- Chain ID 133 (testnet). Testnet RPC: https://testnet.hsk.xyz. Block explorer: https://hashkey.blockscout.com
- Native gas token: HSK. Gas fees as low as 0.1 Gwei. ~2-second block times.
- Compliance-first chain targeting RWA tokenization and institutional DeFi.
- Chainlink integration confirmed — CCIP and price feeds live on the chain.
- $50M Atlas Grant program active — hackathon winners get direct pipeline to grants.

What HashKey Group Has Already Deployed

- Silver RWA tokens — Hong Kong's first regulated silver-backed RWA. Launched March 24, 2026 (one week ago).
- GF Securities tokenized securities — first securities firm to issue tokenized securities in Hong Kong on HashKey Chain.
- Paimon SpaceX tokenized fund — $SPCX deployed on HashKey Chain and BNB Chain.
- HSK listed on HashKey Exchange February 25, 2026 — professional investors only.

Critical Facts From Discord Q&A

CONFIRMED: No active DEXes on HashKey Chain mainnet or testnet. Projects depending on UniswapV2 or any existing DEX liquidity will fail. This means trading-focused projects are impossible. Portfolio management, payment rails, and compliance identity are the real opportunities.

- On-chain KYC is still under development — NexaID (nexaid.gitbook.io) fills this gap and earns extra points.
- HSP (HashKey Settlement Protocol) is live — merchant-qa.hashkeymerchant.com for sandbox, full docs at hashfans.io.
- Testnet tokens available via DM to Benjamin_HSK on Telegram — or the faucet.
- 50% complete projects accepted — roadmap and future plan required. Full MVP still preferred.
- NexaID extra points — not mandatory for ZKID track, but earns bonus with judges.

What the Judges Are Looking For

Judge	What They Care About
HashKey Chain team	Deep ecosystem integration — HSP, NexaID, on-chain RWA usage
HashKey Capital	Viable startup post-hackathon — business model, market, team strength
DGrid AI	Genuine AI agent intelligence — not just API calls dressed as agents
AWS	Cloud-ready deployment, scalable architecture
Kraken	Real trading/settlement logic — not toy demos
EAG / SNZ	Innovation and technical maturity

Prize Strategy — How to Earn $10,000+

First place AI track = $5,000. Plus: HashKey Capital investment pipeline, Atlas Grant access ($50M pool), and ecosystem co-building resources. Non-winning high-potential projects also get reached out to. The real prize is the HashKey Capital relationship — not just the $5K. Strategy: Win AI 1st place + integrate HSP (PayFi bonus) + integrate NexaID (ZKID bonus) = maximum cross-track judge appeal + Atlas Grant eligibility = $10K+ pathway.

PART 2 — THE IDEA: APEX

One Sentence

APEX is an autonomous AI agent that manages on-chain RWA portfolios on HashKey Chain — monitoring Chainlink price feeds, rebalancing positions based on configurable thresholds, and settling withdrawals through HSP payment rails — with no human in the loop after setup.

The Problem

Institutional investors can now hold tokenized RWA assets on HashKey Chain — silver tokens, tokenized securities, structured funds. But managing these positions requires continuous monitoring: tracking prices, checking yield rates, rebalancing when allocations drift, and routing payments when withdrawing yields. Today, all of this is manual. A portfolio manager watches dashboards, executes trades by hand, and routes payments through banks. This is a 19th-century workflow on 21st-century infrastructure.

The problem is structural: smart contracts can hold assets but cannot make decisions. AI can make decisions but cannot execute on-chain. The gap between AI decision and on-chain execution is the problem APEX solves.

The Solution — 4 Autonomous Agent Layers

Layer	What It Does	Technology
Layer 1: Monitor Agent	Watches Chainlink price feeds every 60 seconds. Detects when any RWA asset drifts more than X% from target allocation. Logs all observations to Supabase.	Chainlink oracles on HashKey Chain + Node.js scheduler
Layer 2: Decision Agent	Claude Haiku analyzes the portfolio state, risk thresholds, and market conditions. Outputs a structured rebalancing decision: buy/sell/hold per asset with confidence score and reasoning.	Claude Haiku API (free $5 credit, instant access)
Layer 3: Execution Agent	Submits on-chain transactions to the PortfolioVault smart contract based on the Decision Agent output. Signs transactions using the vault authority keypair.	Hardhat + ethers.js on HashKey Chain testnet
Layer 4: Settlement Agent	When yield is withdrawn from the vault, routes the payment through HSP for compliant cross-border settlement. Handles multi-currency disbursement to institutional recipients.	HSP (HashKey Settlement Protocol) — PayFi integration

Why APEX Cannot Be Built Without HashKey Chain

- **Chainlink on HSK —** Chainlink price feeds are live on HashKey Chain — not available on generic testnets without configuration. APEX needs real oracle data, not mocked prices.
- **HSP exclusive —** HSP only exists in the HashKey ecosystem. The settlement layer for compliant cross-border payment is structurally exclusive to this chain. Remove HSP and APEX becomes a toy with simulated payments.
- **RWA context —** The RWA assets APEX manages — silver tokens, tokenized securities — only exist on HashKey Chain mainnet. Testnet demo uses mock RWA tokens with the same ERC-3643/ERC-20 structure.
- **NexaID bonus —** NexaID identity verification is a HashKey ecosystem product — it earns extra points with judges and is structurally relevant because institutional investors must verify accreditation before accessing RWA vaults.

Why AI Track — Not DeFi or PayFi

DeFi track requires DEX interactions — confirmed impossible since no DEXes exist on chain. PayFi track requires HSP as primary product — APEX uses HSP as one layer, not the whole product. AI track gives maximum creative freedom and aligns perfectly: 'AI-based automatic payment and AI-powered intelligent trading' is the exact description of APEX. The AI track also has the most open judging criteria, meaning a genuinely novel submission stands out more.

> *APEX hits every AI track requirement: AI decision-making (Claude Haiku reasoning layer), automatic payment (HSP settlement), intelligent portfolio management (Chainlink + agent logic), and on-chain execution on HashKey Chain. Cross-track appeal: HSP integration = PayFi bonus. NexaID = ZKID bonus. This is the maximum points configuration.*

PART 3 — STARTUPHACKS.VC FRAMEWORK

Named Persona

Michael, 43 - Head of Digital Assets, Meridian Family Office, Singapore

Michael manages $180M in assets for a Hong Kong-based family office. Six months ago they allocated 8% to tokenized RWAs on HashKey Chain: silver tokens, a tokenized money market fund, and structured securities. Today, monitoring these positions requires three separate dashboards, two manual spreadsheets, and 4 hours of his time every week just to track whether the portfolio is within target allocation.

Last month, the silver token allocation drifted 14% from target while he was traveling. He spotted it 6 days late. The cost was 2.3% in unrealized loss that disciplined rebalancing would have avoided. He manages this with a Bloomberg terminal, three browser tabs, and a Notion doc. There is no autonomous solution for on-chain RWA portfolio management.

Michael would pay $2,000/month for a system that monitors his on-chain positions 24/7, reasons about when to rebalance, executes automatically, and routes yield payments through compliant channels without requiring him to be present.

Michael is in Singapore, Hong Kong, Zurich, Dubai, and New York. He is every institutional digital asset manager who holds tokenized RWA positions on a compliant chain and has no autonomous management layer. HashKey Chain just made his problem bigger by adding more RWA assets — but gave him no management tools.

Problem — Documented and Specific

- Tokenized RWA market hit $14B in December 2024, growing 66% in one year. HashKey Chain is one of the leading issuance venues.
- Every RWA portfolio manager today uses manual dashboards and spreadsheets. Zero autonomous rebalancing products exist on HashKey Chain.
- Cross-border yield distribution still runs through bank wires — 2-5 day settlement, 0.5-1% FX cost. HSP can replace this but no product has integrated it for institutional portfolio yield distribution.
- Chainlink price feeds are live on HashKey Chain but zero products consume them for intelligent decision-making. The oracle data exists unused.

Bottom-Up Market — No TAM

- HashKey Chain has approximately 870K+ wallet addresses and 50+ deployed projects.
- Conservative: 500 institutional portfolio managers holding on-chain RWA positions at $2,000/month = $12M ARR at full penetration of current ecosystem.
- Realistic early: 20 family offices in Hong Kong/Singapore (both major markets HashKey targets) at $2,000/month = $480K ARR in year 1. Achievable given HashKey Capital's direct institutional relationships.

- Path to $2,000 ARR in 60 days: One Michael-type client at $2,000/month, acquired through HashKey Capital's post-hackathon support. The judges ARE the pipeline.

Named Competitors — Strengths First

Competitor	Raised	Real Strengths	APEX Edge
Enzyme Finance	$12M	Best on-chain asset management, mature product, Ethereum/Polygon	Centralized decision-making — no AI agent. No HashKey Chain support. No HSP integration. No RWA context.
dHEDGE	$10M+ raised	Social trading, on-chain managed funds, copy trading	Manual manager decisions. No AI. No HashKey Chain. No compliance layer for institutional RWA.
Coinbase Prime / traditional custodians	$100M+	Institutional trust, regulatory approval, bank integration	Completely off-chain decisions. Manual RWA monitoring. No autonomous execution. No HSP.
Nothing on HashKey Chain	$0	—	Zero autonomous portfolio management products exist on this chain. APEX is first-mover.

Named Moat — Type + Compounding Mechanism

- **Data Moat —** Data moat: Every rebalancing decision APEX makes is logged — Chainlink price at decision time, agent reasoning, outcome. After 10,000 decisions, APEX has the deepest dataset of on-chain RWA price behavior and optimal rebalancing patterns on HashKey Chain. No competitor starting later can replicate this without processing the same history.
- **Switching Cost —** Switching cost moat: Michael's vault configuration, his risk thresholds, his HSP payment routing setup, and his NexaID-verified investor profile all live inside APEX. Moving to a competitor means rebuilding every one of these from scratch.
- **Ecosystem —** Ecosystem moat: The HashKey Capital pipeline from this hackathon gives APEX direct access to their institutional client network. The relationships formed during the hackathon compound — each new institutional client adds more Chainlink data to the decision engine.

Why Now — Verified

- Silver RWA tokens launched on HashKey Chain March 24, 2026 — 8 days ago. GF Securities tokenized securities are live. There are now real on-chain RWA assets to manage and zero autonomous management tools.
- HSP is live in sandbox as of now — merchant-qa.hashkeymerchant.com. The payment settlement layer exists and nobody has built an agent that consumes it.
- Chainlink CCIP went live on HashKey Chain July 2025. The oracle infrastructure is production-ready but zero AI agents consume it on this chain.
- HashKey Capital is a direct investor for hackathon winners. Building now means building into the direct funding pipeline. This window is April 2026 — not a manufactured date.

Revenue Model — Three Stages

Stage	Revenue Stream	Target
Pre-seed (Hackathon → Month 2)	0.10% AUM fee on managed vault value + $500/month base subscription per vault. No sales call — HashKey Capital makes the intro.	$2,000 ARR: one client at $2,000/month. Prove agent executes correctly and HSP payment lands.
Seed (Months 3-12)	0.10% AUM fee scales with TVL. API access for other dApps wanting AI portfolio management ($500-2,000/month). White-label per institutional client.	$480K ARR: 20 family offices at $2,000/month average.
Series A (Months 12-36)	Revenue share with HashKey Capital for institutional referrals (2% of AUM fee). Multi-chain expansion — APEX on other compliant chains.	$2M+ ARR: 80+ clients, multi-chain, revenue share compound.

GTM Machine — 5 Components, Zero Cost

1. Segment + Purchase Trigger

Primary segment: institutional digital asset managers at Hong Kong and Singapore family offices managing on-chain RWA positions. Purchase trigger: a rebalancing event they caught late, or a yield payment routed through a bank that took 3 days. HashKey Capital has direct relationships with exactly these clients.

2. Acquisition Channels (Ranked)

1. **Channel 1:** HashKey Capital direct introduction — winning this hackathon puts APEX in front of HashKey Capital's institutional client network immediately. This is the highest-value distribution channel possible. One warm introduction from a judge is worth more than 1,000 cold emails.
2. **Channel 2:** HashKey Chain Eco Grants ($50M Atlas Grant program) — apply immediately post-hackathon. Grant recipients get technical guidance and ecosystem co-building resources, not just funding.

3. **Channel 3:** Web3 Festival Hong Kong April 23 — the hackathon final pitch is at the Web3 Festival. Every institutional investor attending is a potential Michael. Demo APEX live on stage.
4. **Channel 4:** AWS co-marketing — AWS is a judge. A product built on their cloud infrastructure with a story about institutional AI is exactly what they showcase in their FinTech customer stories.
5. **Channel 5:** HashFans developer community — post a technical breakdown of the agent architecture. Other developers integrate APEX as a module.

3. Conversion Funnel

6. Michael sees APEX at the Web3 Festival pitch (or via HashKey Capital intro).
7. He tries the live demo: connects wallet, configures risk thresholds, watches Monitor Agent pull live Chainlink data, watches Decision Agent reason about the portfolio, watches Execution Agent simulate a rebalancing transaction.
8. Magic moment: Decision Agent outputs reasoning in plain English — 'Silver token is 12% over target. Recommend reduce by 8%, redistribute to MMF. Confidence: 87%. Reason: Silver volatility rising per 30-day Chainlink data.' Michael sees the AI think in the way he would think.
9. Michael signs up. Vault deployed. HSP payment routing configured for his Singapore bank account.
10. First autonomous rebalancing event fires without Michael doing anything. He receives a Telegram notification: 'APEX rebalanced your portfolio at 14:32 UTC. Silver -8%, MMF +8%. New allocation within target. TX: 0x...'

4. Retention

- Weekly portfolio performance report emailed to Michael — Chainlink price movements, decisions made, yield earned, HSP payments processed.
- Every autonomous action is logged with the agent's reasoning. Michael can audit every decision at any time. This is the compliance deliverable institutional investors need.

5. Seed Raise Story

Enzyme Finance raised $12M managing on-chain portfolios with manual human managers. APEX removes the human with an AI agent specifically calibrated for HashKey Chain's RWA ecosystem — the fastest-growing compliant tokenized asset ecosystem in Asia. We manage $X in assets, process $Y in yield payments through HSP monthly, and serve Z institutional vaults. We raise $1M pre-seed to onboard the first 20 institutional clients and reach $480K ARR.

PART 4 — TECHNICAL ARCHITECTURE

HashKey Chain Configuration

Parameter	Value
Testnet RPC	https://testnet.hsk.xyz
Chain ID	133 (testnet)
Block Explorer	https://hashkey.blockscout.com
Native Token	HSK (gas)
Testnet USDC	Request from bridge: bridge.hsk.xyz (ETH Sepolia → HSK testnet)
Testnet HSK faucet	DM @Benjamin_HSK on Telegram with wallet address
EVM Tools	Standard Hardhat/Foundry/ethers.js — EVM compatible
Chainlink Feeds	Deployed on HashKey Chain — use standard Chainlink AggregatorV3Interface

Smart Contracts — PortfolioVault.sol

Three contracts. Deploy to testnet. Verified on blockscout.

Contract 1: PortfolioVault.sol (Core)

- Holds deposited RWA tokens (mock ERC-20 tokens representing silver, MMF, securities)
- Stores portfolio target allocations per asset: { silver: 30%, mmf: 50%, securities: 20% }
- Stores rebalancing thresholds: drift > 5% triggers rebalance eligibility
- Records all decisions on-chain with timestamp + agent reasoning hash
- executeRebalance(asset, direction, amount) — callable only by authorized APEX agent address
- withdrawYield(recipient, amount, hspRef) — withdraws yield to recipient, stores HSP reference
- Events: PortfolioRebalanced, YieldWithdrawn, AllocationUpdated, DecisionLogged

Contract 2: AgentRegistry.sol

- Stores authorized agent addresses and their permissions
- Links agent addresses to NexaID-verified identity (via NexaID attestation hash)
- setAgentPermission(address, permission) — callable by vault owner only
- verifyAgent(address) — returns true if agent is authorized and NexaID verified

Contract 3: DecisionLog.sol

- Immutable on-chain record of every agent decision
- logDecision(portfolioId, decisionType, assetAddress, confidence, reasoningHash)
- Reasoning stored as keccak256 hash of the full Claude response (off-chain stored in Supabase)
- This is the compliance audit trail institutional investors and regulators need

Agent Architecture — Node.js

Monitor Agent (runs every 60 seconds)

Reads Chainlink price feeds for each asset in the vault. Computes current allocation percentages. Compares against target allocations. If any asset drifts > threshold: emits REBALANCE_NEEDED event to Decision Agent.

Decision Agent (triggered by Monitor)

Calls Claude Haiku with: current prices, drift amounts, risk thresholds, historical volatility. Receives structured JSON: { action: 'rebalance', asset: '0x...', direction: 'reduce', amount: 8, confidence: 87, reasoning: '...' }. Logs reasoning to Supabase. Passes decision to Execution Agent.

Execution Agent (triggered by Decision)

Builds and signs a transaction to call PortfolioVault.executeRebalance with the Decision Agent output. Also calls DecisionLog.logDecision with the decision hash. Sends transaction to HashKey Chain testnet. Monitors for confirmation. Sends Telegram notification on completion.

Settlement Agent (triggered by yield withdrawal)

When vault owner requests yield withdrawal, calls HSP API to initiate a payment to the configured bank/wallet. HSP handles the cross-border settlement. Agent stores the HSP reference ID on-chain in the withdrawal record.

HSP Integration (PayFi Layer)

> *HSP (HashKey Settlement Protocol) is used for yield payment settlement. When Michael withdraws quarterly yield, instead of a direct on-chain transfer, APEX routes it through HSP — which handles currency conversion, compliance verification, and settlement to his Singapore bank account. This is the PayFi track bonus.*

- HSP sandbox: merchant-qa.hashkeymerchant.com/en/dashboard/login
- Full docs: hashfans.io/assets/merchant-docs-all-in-one.pdf

- HSP does NOT manage funds — only handles payment message transmission and status sync
- Integration: APEX calls HSP API to create a payment request after yield is withdrawn from vault
- HSP returns a reference ID — stored on-chain in the withdrawal transaction

NexaID Integration (ZKID Bonus Layer)

- NexaID docs: nexaid.gitbook.io/nexaid/build-on-nexaid/extension/prerequisites
- Institutional investors verify their accredited status via NexaID before vault access is granted
- NexaID issues a ZK proof: 'this wallet belongs to an accredited investor' without revealing identity
- AgentRegistry.sol stores the NexaID attestation hash alongside the vault owner address
- Demo flow: wallet connects → NexaID verification prompt → ZK proof generated → agent unlocks vault access

Free Tech Stack — All Verified

Tool	How to Get	Use In APEX	Cost
Hardhat + ethers.js	npm install --save-dev hardhat	Contract deploy + agent txns	**Free**
Claude Haiku API	console.anthropic.com — instant $5 credit	Decision agent brain	**Free**
Supabase	supabase.com — GitHub login	Decision logs + agent state	**Free tier**
Vercel	vercel.com — GitHub login	Frontend deploy	**Free tier**
Next.js + Tailwind	npx create-next-app@latest	Dashboard UI	**Free**
Lovable	lovable.dev — existing account	UI mockup to start	**Free tier**
HSP sandbox	merchant-qa.hashkeymerchant.com	Payment settlement layer	**Free**
NexaID	nexaid.gitbook.io — follow prereqs	ZK identity verification	**Free**
Chainlink AggregatorV3	On HashKey Chain — no key needed	Price feeds for Monitor Agent	**Free**
Telegram Bot API	t.me/BotFather — instant	Agent status notifications	**Free**

PART 5 — ENV KEYS + SETUP

Deadline is April 15 23:59 GMT+8. Today is April 1. That's 14 days. Get all keys TODAY so Day 1 of coding starts clean.

Step 1 — Get Testnet HSK (Gas)

11. Open Telegram. Find @Benjamin_HSK (organizer).
12. Send: 'Hi Benjamin — I'm building APEX for the AI track. Can I get some testnet HSK for deployment and testing? Wallet: [your wallet address]'
13. Usually responds within a few hours.
14. Alternatively: DM the general Telegram group mentioned in the hackathon page.

Step 2 — Get Testnet Tokens (Bridge)

15. Go to bridge.hsk.xyz
16. Connect wallet. Select ETH Sepolia → HashKey Chain Testnet.
17. Bridge a small amount of Sepolia ETH to get testnet ETH on HashKey Chain.
18. For USDC on testnet: use the same bridge if USDC is available, otherwise deploy your own mock ERC-20 (5 min in Remix).

Step 3 — Claude API Key (Decision Agent Brain)

19. Go to console.anthropic.com
20. Sign up with any email — instant $5 credit.
21. API Keys → Create Key → copy.

Env var: ANTHROPIC_API_KEY=sk-ant-...

Cost estimate for hackathon: Claude Haiku is $0.25/M input tokens. APEX makes ~100 decisions/day in demo mode. Well under $1 total.

Step 4 — Supabase (Decision Logs + Agent State)

22. Go to supabase.com → GitHub login → New Project.
23. Run this SQL in SQL Editor:

```
CREATE TABLE decisions (
  id uuid primary key default gen_random_uuid(),
  portfolio_id text not null,
  asset_address text,
  decision_type text,  -- rebalance / hold / emergency_stop
  direction text,      -- increase / reduce
  amount_pct numeric,
```

```
  confidence numeric,
  reasoning text,      -- full Claude output
  reasoning_hash text, -- keccak256 stored on-chain
  chainlink_price numeric,
  tx_hash text,
  executed_at timestamptz default now()
);

CREATE TABLE portfolios (
  id uuid primary key default gen_random_uuid(),
  vault_address text unique not null,
  owner_wallet text not null,
  nexaid_hash text,    -- NexaID attestation
  assets jsonb,        -- [{address, symbol, target_pct}]
  thresholds jsonb,    -- {drift_pct: 5, max_single: 40}
  hsp_config jsonb,    -- HSP payment routing details
  created_at timestamptz default now()
);

CREATE TABLE agent_logs (
  id uuid primary key default gen_random_uuid(),
  portfolio_id text,
  action text,
  details jsonb,
  created_at timestamptz default now()
);

ALTER TABLE decisions ENABLE ROW LEVEL SECURITY;
ALTER TABLE portfolios ENABLE ROW LEVEL SECURITY;
ALTER TABLE agent_logs ENABLE ROW LEVEL SECURITY;
CREATE POLICY "open" ON decisions FOR ALL USING (true);
CREATE POLICY "open" ON portfolios FOR ALL USING (true);
CREATE POLICY "open" ON agent_logs FOR ALL USING (true);
```

24. Copy: Project URL, anon key, service_role key.

Env vars: NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_KEY

Step 5 — HSP Sandbox Setup

25. Go to merchant-qa.hashkeymerchant.com/en/dashboard/login
26. Register for a sandbox account — follow the PDF guide at hashfans.io/assets/merchant-docs-all-in-one.pdf
27. Create a test merchant. Note: HSP handles payment message transmission, not fund custody.
28. Get: API key, Merchant ID, sandbox endpoint URL.

Env vars: HSP_API_KEY, HSP_MERCHANT_ID, HSP_BASE_URL

Step 6 — NexaID Setup

29. Go to nexaid.gitbook.io/nexaid/build-on-nexaid/extension/prerequisites
30. Follow prerequisites to get a Template ID for identity verification.

31. Get: API key or Template ID.

Env var: NEXAID_TEMPLATE_ID, NEXAID_API_KEY (if required)

Step 7 — Telegram Bot (Agent Notifications)

32. Open Telegram → @BotFather → /newbot → follow prompts.
33. Copy the Bot Token.

Env vars: TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID (your chat ID — get from @userinfobot)

Step 8 — Vercel Deploy

34. vercel.com → GitHub login → free tier.
35. Connect the GitHub repo when ready to deploy.
36. Add all env vars in Vercel dashboard → Settings → Environment Variables.

Complete .env Template

```
# ── HashKey Chain ──
HASHKEY_TESTNET_RPC=https://testnet.hsk.xyz
HASHKEY_CHAIN_ID=133
PRIVATE_KEY=your_funded_deployer_key
VAULT_CONTRACT=  # fill after deploy
AGENT_REGISTRY=  # fill after deploy
DECISION_LOG=    # fill after deploy

# ── Chainlink (no key needed — reads on-chain) ──
SILVER_PRICE_FEED=0x...   # find at docs.chain.link for HSK testnet
MMF_PRICE_FEED=0x...
SECURITIES_PRICE_FEED=0x...

# ── Claude API (Decision Agent) ──
ANTHROPIC_API_KEY=sk-ant-...

# ── Supabase ──
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_KEY=eyJ...

# ── HSP (PayFi Settlement) ──
HSP_API_KEY=
HSP_MERCHANT_ID=
HSP_BASE_URL=https://merchant-qa.hashkeymerchant.com

# ── NexaID (ZKID Bonus) ──
NEXAID_TEMPLATE_ID=
NEXAID_API_KEY=

# ── Telegram Notifications ──
```

```
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=
```

PART 6 — 14-DAY BUILD PLAN

Submission deadline: April 15 23:59 GMT+8. Demo pre-screening: April 14. Today is April 1. 14 days total. Day 1 is about environment. Day 14 is submission.

Day	Focus	Deliverable	Exit Gate
Day 1	**Environment**	Get HSK testnet from Benjamin. Set up Supabase, Anthropic key, Telegram bot. Scaffold Hardhat project with HashKey testnet config.	All env vars in .env. wallet has testnet HSK.
Day 2	**Smart Contracts**	Write PortfolioVault.sol + AgentRegistry.sol + DecisionLog.sol. Deploy to testnet.	3 contracts live on hashkey.blockscout.com.
Day 3	**Mock RWA tokens**	Deploy 3 mock ERC-20 tokens (Silver, MMF, Securities). Fund them into the vault. Verify all 3 on blockscout.	Vault holds mock tokens. Can read balances.
Day 4	**Monitor Agent**	Node.js service polls Chainlink price feeds every 60s. Computes portfolio drift. Logs to Supabase.	Monitor Agent runs 30 min, logs 30 events to Supabase.
Day 5	**Decision Agent**	Claude Haiku prompt engineering. Structured JSON output. Decision logged to Supabase + hash stored on-chain.	Decision Agent produces valid structured JSON from Claude.
Day 6	**Execution Agent**	Reads Decision Agent output. Builds and signs transactions to PortfolioVault. Submits to testnet.	End-to-end: monitor detects drift → Claude decides → vault executes. Show TX on explorer.
Day 7	**HSP Integration**	Implement HSP API call for yield payment routing. Test with sandbox. Store HSP reference on-chain.	HSP payment created in sandbox. Reference saved on-chain.
Day 8	**NexaID Integration**	Add NexaID verification flow to AgentRegistry. Vault gated by NexaID proof.	NexaID verified wallet can access vault. Unverified wallet blocked.
Day 9	**Telegram Bot**	Agent sends Telegram message on every decision: 'APEX rebalanced. Silver -8%, MMF +8%. TX: 0x...'	Telegram notification fires on every execution.
Day 10-11	**Frontend (Lovable)**	Build dashboard: portfolio allocation chart, decision log table, agent status, vault configuration. Push to GitHub and clone.	Dashboard shows live data from Supabase + HashKey Chain.
Day 12	**Frontend integration**	Wire frontend to contracts (ethers.js). Connect wallet. Real-time decision feed.	Full demo cycle works end-to-end from browser.
Day 13	**Testing + Polish**	Run 10 end-to-end cycles. Fix any bugs. Polish README and demo docs.	10 clean cycles with TX hashes on blockscout.
Day 14	**Submit (Apr 15)**	Record 3-minute video. Complete DoraHacks submission form. Submit by 23:59 GMT+8.	Submission confirmed on DoraHacks.

Cut List — What Gets Dropped If Time Runs Short

Never cut: 4-agent pipeline (Monitor → Decision → Execution → Settlement), Chainlink price feeds, HSP integration, blockchain TX on blockscout. These are the judging criteria. Cut if needed: NexaID verification (bonus not required), multi-asset support (start with 2 assets), historical analytics charts, multi-vault management.

PART 7 — CLAUDE CODE PROMPTS

7 prompts. Run in order. Wait for each to complete before the next. These stack on each other.

Prompt 0 — Research First

```
Before writing any code, fetch and read these docs:

1. https://docs.hsk.xyz/developerquickstart
2. https://docs.hsk.xyz/
3. https://nexaid.gitbook.io/nexaid/build-on-nexaid/extension/prerequisites
4. https://hashfans.io/assets/merchant-docs-all-in-one.pdf

Also search the web for:
- "Chainlink price feed AggregatorV3Interface HashKey Chain testnet 2025"
- "HashKey Chain RWA ERC-3643 token standard 2025"
- "HSP HashKey Settlement Protocol payment API integration"
- "ethers.js HashKey Chain testnet deploy contract 2025"

After reading, tell me:
1. The exact HashKey Chain testnet RPC URL and Chain ID
2. Whether Chainlink price feeds are available on HashKey testnet
   and what the feed addresses are (or how to find them)
3. What NexaID requires for integration (Template ID process)
4. How HSP payment API is called (endpoint + auth)

Do NOT write any code yet.
```

Prompt 1 — Hardhat Setup + Smart Contracts

```
Set up the APEX project — autonomous RWA portfolio management
agent on HashKey Chain.

INSTALL:
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox
npm install @openzeppelin/contracts
npm install @chainlink/contracts
npm install ethers dotenv

HARDHAT CONFIG (hardhat.config.ts):
networks: {
  hashkeyTestnet: {
    url: "https://testnet.hsk.xyz",
    chainId: 133,
    accounts: [process.env.PRIVATE_KEY!]
  }
}

WRITE THESE 4 CONTRACTS:

_____________________________________________
contracts/MockRWA.sol (deploy 3 instances: Silver, MMF, Securities)
_____________________________________________
```

```
// Simple ERC-20 with mint function for testing
// Symbol/name passed in constructor
// Anyone can mint in testnet (for demo seeding)

──────────────────────────────────────────────────
contracts/PortfolioVault.sol
──────────────────────────────────────────────────
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

State variables:
  address public agent;       // APEX agent address (authorized to rebalance)
  address public owner;       // vault owner (Michael)
  address[] public assets;    // tracked asset addresses

  struct AssetConfig {
    uint256 targetPct;        // target allocation % (e.g., 30 = 30%)
    uint256 driftThreshold;   // trigger if drift > this (e.g., 5 = 5%)
    bool active;
  }
  mapping(address => AssetConfig) public assetConfigs;

  struct Withdrawal {
    address recipient;
    uint256 amount;
    address token;
    bytes32 hspRef;           // HSP payment reference
    uint256 timestamp;
  }
  Withdrawal[] public withdrawals;

Key functions:
  deposit(address token, uint256 amount) — owner deposits tokens

  executeRebalance(address token, bool isIncrease, uint256 amount)
    onlyAgent modifier
    Updates token balance tracking
    Emits RebalancedExecuted(token, isIncrease, amount, block.timestamp)

  withdrawYield(address token, uint256 amount, bytes32 hspRef)
    onlyOwner modifier
    Transfers tokens out
    Records withdrawal with HSP reference
    Emits YieldWithdrawn(recipient, amount, hspRef)

  getCurrentAllocations() view returns (address[], uint256[])
    Returns all tracked assets and their current balance percentages

Events:
  RebalanceExecuted(address token, bool isIncrease, uint256 amount, uint256
timestamp)
  YieldWithdrawn(address recipient, uint256 amount, bytes32 hspRef)
  AgentUpdated(address newAgent)

──────────────────────────────────────────────────
contracts/DecisionLog.sol
──────────────────────────────────────────────────
State:
  struct Decision {
```

```
    bytes32 portfolioId;
    address asset;
    string decisionType;     // "rebalance" / "hold" / "emergency"
    bool isIncrease;
    uint256 amount;
    uint8 confidence;        // 0-100
    bytes32 reasoningHash;   // keccak256 of full Claude response
    uint256 timestamp;
  }
  Decision[] public decisions;

Function:
  logDecision(...) onlyAgent — stores decision on-chain
  getDecisionCount() view returns (uint256)
  getDecision(uint256 index) view returns (Decision)

_____________________________________________
contracts/AgentRegistry.sol
_____________________________________________
  mapping(address => bool) public authorizedAgents;
  mapping(address => bytes32) public nexaidHash;  // NexaID attestation

  registerAgent(address agent) onlyOwner
  verifyNexaId(address user, bytes32 hash) onlyOwner — stores hash
  isAuthorized(address agent) view returns (bool)

DEPLOY SCRIPT (scripts/deploy.ts):
Deploy order:
1. MockRWA x3 (Silver, MMF, Securities)
2. PortfolioVault
3. DecisionLog
4. AgentRegistry

After deploy, seed vault:
- Mint 1000 tokens of each MockRWA
- Deposit all into PortfolioVault
- Set target allocations: Silver 30%, MMF 50%, Securities 20%
- Set drift threshold: 5%
- Set agent address to your deployer address

Run: npx hardhat run scripts/deploy.ts --network hashkeyTestnet
Show me: all 5 contract addresses on hashkey.blockscout.com
```

Prompt 2 — Monitor Agent (Chainlink + Drift Detection)

```
Build the Monitor Agent. This runs every 60 seconds and detects
portfolio drift using Chainlink price feeds on HashKey Chain.

FILE: agent/monitorAgent.ts

Install: npm install @chainlink/contracts ethers node-cron

import { ethers } from 'ethers'
import cron from 'node-cron'
import { supabase } from './supabase'
```

```
const provider = new ethers.JsonRpcProvider(process.env.HASHKEY_TESTNET_RPC)
const vaultAbi = [...] // import from artifacts
const vault = new ethers.Contract(VAULT_ADDRESS, vaultAbi, provider)

// Chainlink price feed ABI (standard AggregatorV3Interface)
const feedAbi = [
  "function latestRoundData() external view returns (uint80, int256, uint256,
uint256, uint80)"
]

// NOTE: If Chainlink feeds aren't available on HashKey testnet,
// implement a MockPriceFeed.sol that returns configurable prices
// and deploy it. For the demo, prices can be updated manually
// to simulate drift.

async function checkPortfolio() {
  // 1. Read current allocations from vault
  const [assets, balances] = await vault.getCurrentAllocations()

  // 2. For each asset, read price from Chainlink feed
  // If no Chainlink feed exists for testnet, use MockPriceFeed
  const prices: Record<string, number> = {}
  for (const asset of assets) {
    const feedAddress = PRICE_FEEDS[asset]
    if (feedAddress) {
      const feed = new ethers.Contract(feedAddress, feedAbi, provider)
      const [,price] = await feed.latestRoundData()
      prices[asset] = Number(price) / 1e8  // Chainlink uses 8 decimals
    }
  }

  // 3. Compute current USD values and allocation percentages
  const totalValue = assets.reduce((sum, asset, i) =>
    sum + (Number(balances[i]) * (prices[asset] || 1)), 0)

  const allocations: PortfolioState = assets.map((asset, i) => ({
    address: asset,
    balance: Number(balances[i]),
    price: prices[asset] || 1,
    usdValue: Number(balances[i]) * (prices[asset] || 1),
    currentPct: (Number(balances[i]) * (prices[asset] || 1) / totalValue) * 100,
    targetPct: TARGET_ALLOCATIONS[asset],
    drift: Math.abs(currentPct - targetPct)
  }))

  // 4. Log to Supabase agent_logs table
  await supabase.from('agent_logs').insert({
    portfolio_id: VAULT_ADDRESS,
    action: 'monitor_check',
    details: { allocations, totalValue, timestamp: new Date().toISOString() }
  })

  // 5. If any asset drift > threshold, emit to Decision Agent
  const drifted = allocations.filter(a => a.drift > DRIFT_THRESHOLD)
  if (drifted.length > 0) {
    console.log('[APEX Monitor] Drift detected:', drifted.map(a =>
      a.address + ': ' + a.drift.toFixed(2) + '%'
    ))
    // Call Decision Agent
    await triggerDecisionAgent(allocations, drifted)
```

```
  } else {
    console.log('[APEX Monitor] Portfolio within thresholds. All clear.')
  }
}

// Run every 60 seconds
cron.schedule('* * * * *', checkPortfolio)

// For demo: add a /trigger-drift endpoint that artificially
// changes mock price to simulate a drift event immediately
// This is for the live demo video

console.log('[APEX Monitor] Started. Checking every 60s.')
checkPortfolio() // initial check

Test: Run for 5 minutes. Show me the Supabase agent_logs
table with 5+ rows.
```

Prompt 3 — Decision Agent (Claude Haiku)

```
Build the Decision Agent. This calls Claude Haiku to reason
about whether to rebalance and what action to take.

FILE: agent/decisionAgent.ts

import Anthropic from '@anthropic-ai/sdk'
import { ethers } from 'ethers'
import { supabase } from './supabase'

const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY })

export async function triggerDecisionAgent(
  allocations: PortfolioState[],
  driftedAssets: PortfolioState[]
) {
  // Build prompt for Claude
  const portfolioSummary = allocations.map(a =>
    `${a.symbol}: current ${a.currentPct.toFixed(1)}%, target ${a.targetPct}%,
drift ${a.drift.toFixed(1)}%`
  ).join('\n')

  const prompt = `You are APEX, an autonomous RWA portfolio management agent on
HashKey Chain.

Current portfolio state:
${portfolioSummary}

Drifted assets requiring attention:
${driftedAssets.map(a => `${a.symbol}: ${a.drift.toFixed(1)}% drift`).join('\n')}

Rules:
- Maximum single asset allocation: 60%
- Minimum single asset allocation: 10%
- Rebalance only when drift > 5%
- Prefer smallest action that restores allocation
```

```
- Risk level: Conservative (institutional portfolio)

You MUST respond with ONLY valid JSON in this exact format:
{
  "action": "rebalance" or "hold",
  "transactions": [
    {
      "asset": "asset_address_here",
      "direction": "increase" or "reduce",
      "amount_pct": 8,
      "reasoning": "one sentence why"
    }
  ],
  "overall_confidence": 87,
  "summary": "one sentence summary of decision"
}`

  const message = await anthropic.messages.create({
    model: 'claude-haiku-4-5-20251001',
    max_tokens: 1000,
    messages: [{ role: 'user', content: prompt }]
  })

  const responseText = message.content[0].type === 'text'
    ? message.content[0].text : ''

  // Parse JSON safely
  let decision
  try {
    const clean = responseText.replace(/```json|```/g, '').trim()
    decision = JSON.parse(clean)
  } catch {
    console.error('[APEX Decision] Failed to parse Claude response:',
responseText)
    return
  }

  // Compute reasoning hash for on-chain storage
  const reasoningHash = ethers.keccak256(
    ethers.toUtf8Bytes(responseText)
  )

  // Store full decision in Supabase
  await supabase.from('decisions').insert({
    portfolio_id: VAULT_ADDRESS,
    decision_type: decision.action,
    reasoning: responseText,
    reasoning_hash: reasoningHash,
    confidence: decision.overall_confidence,
    details: decision
  })

  console.log('[APEX Decision]', decision.summary)
  console.log('[APEX Decision] Confidence:', decision.overall_confidence + '%')

  // If action = rebalance, pass to Execution Agent
  if (decision.action === 'rebalance') {
    await triggerExecutionAgent(decision, reasoningHash)
  }
}
```

```
Test: Call triggerDecisionAgent with a mock portfolio showing
Silver at 42% (target 30%) and MMF at 38% (target 50%).
Show me the Claude JSON output and confirm it's valid.
```

Prompt 4 — Execution Agent + DecisionLog

```
Build the Execution Agent. This reads the Decision Agent output
and executes the rebalancing on HashKey Chain.

FILE: agent/executionAgent.ts

import { ethers } from 'ethers'
import { supabase } from './supabase'
import { sendTelegramNotification } from './telegram'

const provider = new ethers.JsonRpcProvider(process.env.HASHKEY_TESTNET_RPC)
const wallet = new ethers.Wallet(process.env.PRIVATE_KEY!, provider)
const vault = new ethers.Contract(VAULT_ADDRESS, vaultAbi, wallet)
const decisionLog = new ethers.Contract(DECISION_LOG_ADDRESS, decisionLogAbi,
wallet)

export async function triggerExecutionAgent(
  decision: Decision,
  reasoningHash: string
) {
  console.log('[APEX Execution] Executing:', decision.summary)

  for (const txn of decision.transactions) {
    try {
      // 1. Execute rebalance on PortfolioVault
      const tx = await vault.executeRebalance(
        txn.asset,
        txn.direction === 'increase',
        ethers.parseUnits(txn.amount_pct.toString(), 18)
      )

      const receipt = await tx.wait()
      console.log('[APEX Execution] TX confirmed:', receipt.hash)

      // 2. Log decision on-chain to DecisionLog contract
      const logTx = await decisionLog.logDecision(
        ethers.keccak256(ethers.toUtf8Bytes(VAULT_ADDRESS)), // portfolioId
        txn.asset,
        decision.action,
        txn.direction === 'increase',
        txn.amount_pct,
        decision.overall_confidence,
        reasoningHash
      )
      await logTx.wait()

      // 3. Update Supabase with TX hash
      await supabase.from('decisions').update({
        tx_hash: receipt.hash,
        executed_at: new Date().toISOString()
```

```
      }).eq('reasoning_hash', reasoningHash)

      // 4. Send Telegram notification
      await sendTelegramNotification(
        `✅ APEX Rebalanced Portfolio\n` +
        `Asset: ${txn.asset}\n` +
        `Action: ${txn.direction} by ${txn.amount_pct}%\n` +
        `Confidence: ${decision.overall_confidence}%\n` +
        `Reason: ${txn.reasoning}\n` +
        `TX: https://hashkey.blockscout.com/tx/${receipt.hash}`
      )

    } catch (err) {
      console.error('[APEX Execution] Failed:', err)
      await supabase.from('agent_logs').insert({
        portfolio_id: VAULT_ADDRESS,
        action: 'execution_failed',
        details: { error: String(err), transaction: txn }
      })
    }
  }
}

FILE: agent/telegram.ts

export async function sendTelegramNotification(message: string) {
  const url =
`https://api.telegram.org/bot${process.env.TELEGRAM_BOT_TOKEN}/sendMessage`
  await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      chat_id: process.env.TELEGRAM_CHAT_ID,
      text: message,
      parse_mode: 'Markdown'
    })
  })
}

FULL END-TO-END TEST:
1. Run monitorAgent.ts
2. Manually call MockPriceFeed to push Silver price up 20% (simulates drift)
3. Monitor Agent detects drift
4. Decision Agent calls Claude → JSON output
5. Execution Agent submits TX to HashKey Chain testnet
6. DecisionLog records on-chain
7. Telegram notification fires
8. Supabase updated with TX hash

Show me:
- TX hash on hashkey.blockscout.com
- Claude reasoning JSON from Supabase
- Telegram notification screenshot
```

Prompt 5 — HSP Settlement Integration

```
Integrate HSP (HashKey Settlement Protocol) for yield payment
```

```
settlement — the PayFi track bonus layer.

Read the HSP docs: hashfans.io/assets/merchant-docs-all-in-one.pdf

FILE: agent/settlementAgent.ts

HSP is the HashKey Settlement Protocol. It handles payment
message transmission, verification, and status synchronization.
It does NOT hold funds — it only handles the payment messaging layer.

async function initiateHSPPayment(params: {
  amount: number,
  currency: string,    // "USDC" or "HSK"
  recipient: string,   // wallet or bank account
  reference: string,   // internal reference
  vaultId: string
}): Promise<string> {  // returns HSP reference ID

  // Step 1: Create payment request via HSP API
  const response = await fetch(`${process.env.HSP_BASE_URL}/api/payment/create`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.HSP_API_KEY}`,
      'X-Merchant-ID': process.env.HSP_MERCHANT_ID!,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      merchantId: process.env.HSP_MERCHANT_ID,
      amount: params.amount,
      currency: params.currency,
      recipient: params.recipient,
      reference: params.reference,
      description: `APEX yield distribution - vault ${params.vaultId}`
    })
  })

  // NOTE: If HSP sandbox API shape differs from above,
  // adapt to whatever the actual merchant-docs-all-in-one.pdf specifies.
  // The key is: create payment → get reference ID → store on-chain.

  const result = await response.json()
  return result.referenceId || result.paymentId || result.id
}

// Called when vault owner requests yield withdrawal
export async function settleYieldPayment(
  vaultContract: ethers.Contract,
  token: string,
  amount: number,
  recipient: string
) {
  console.log('[APEX Settlement] Initiating HSP payment...')

  // 1. Create HSP payment request
  const hspRef = await initiateHSPPayment({
    amount,
    currency: 'USDC',
    recipient,
    reference: `apex-${Date.now()}`,
    vaultId: VAULT_ADDRESS
```

```
  })

  // 2. Execute withdrawal from vault with HSP reference
  const hspRefBytes32 = ethers.keccak256(ethers.toUtf8Bytes(hspRef))
  const tx = await vaultContract.withdrawYield(
    token,
    ethers.parseUnits(amount.toString(), 18),
    hspRefBytes32
  )
  const receipt = await tx.wait()

  // 3. Log to Supabase
  await supabase.from('agent_logs').insert({
    portfolio_id: VAULT_ADDRESS,
    action: 'yield_settled',
    details: { hspRef, txHash: receipt.hash, amount, recipient }
  })

  console.log('[APEX Settlement] Complete. HSP Ref:', hspRef)
  console.log('[APEX Settlement] TX:', receipt.hash)

  return { hspRef, txHash: receipt.hash }
}

// API endpoint to trigger yield withdrawal
// POST /api/withdraw-yield { token, amount, recipient }
// Calls settleYieldPayment and returns the result

Test: Create a payment in HSP sandbox.
Show me: the HSP reference ID and the on-chain TX with hspRef stored.
```

Prompt 6 — Frontend (Lovable + VSCode Integration)

```
Build the APEX dashboard. Mobile-first. Professional dark theme
matching institutional finance aesthetics.

LOVABLE PROMPT (build UI first in Lovable, then push to GitHub):

Build the APEX institutional portfolio dashboard with these pages:

DESIGN: Dark theme. Deep navy (#0F172A) background.
Blue (#1A56DB) accents. Green (#059669) for positive actions.
Professional, institutional. No bright colors or crypto aesthetics.
Font: Inter or similar clean sans-serif.

PAGE 1: Dashboard (/)
Top row — 4 KPI cards:
  Total AUM ($1,240,000 USDC)
  Active Agents (4 — Monitor/Decision/Execution/Settlement)
  Today's Decisions (3)
  Yield This Month ($12,400 USDC)

Portfolio allocation chart (donut chart):
  Silver 28% / MMF 52% / Securities 20%
  Show drift from target as colored indicators: green=on-target, amber=near
threshold, red=rebalancing needed
```

```
Live Agent Feed (real-time list, newest first):
  Each entry: timestamp, agent name badge (Monitor/Decision/Execution/Settlement),
action, details
  Color-coded by agent type

Recent Decisions table:
  Columns: Time | Asset | Action | Amount | Confidence | Reasoning | TX Link
  TX Link: clickable to hashkey.blockscout.com

PAGE 2: Vault Configuration (/config)
Asset Allocation Sliders:
  Silver [===30%===] target: 30%, threshold: 5%
  MMF [=======50%=======] target: 50%, threshold: 5%
  Securities [==20%==] target: 20%, threshold: 5%

NexaID Status: Verified ✓ (green badge)
HSP Payment Config: Singapore bank account ending 4821

Vault Address: 0x... (clickable to explorer)
Agent Address: 0x... (clickable to explorer)

PAGE 3: Decision Log (/decisions)
Full table of all decisions from Supabase:
  Time | Claude Reasoning | Confidence | Action | TX Hash | On-Chain Hash

Each row expandable to show full Claude response text.
"Verified on-chain" badge showing the reasoning hash matches blockchain.

PAGE 4: Settlement (/settlement)
Yield withdrawal form:
  Token: [dropdown: Silver | MMF | Securities]
  Amount: [input]
  Recipient: [wallet address input]
  [Initiate HSP Settlement] button

Settlement history table:
  Date | Amount | Asset | HSP Reference | TX Hash | Status

After Lovable builds the UI:
1. Push to GitHub: git add . && git commit -m "feat: APEX dashboard UI" && git
push
2. Come back here

I will then wire the frontend to the smart contracts and Supabase.
```

Prompt 7 — Wire Frontend + Final Integration

```
Read all frontend files in the cloned Lovable repo first.
Then wire everything to live data.

Install:
npm install ethers @supabase/supabase-js

FILE: lib/contracts.ts
```

```
- Configure ethers.js for HashKey Chain testnet (chainId 133)
- Export vault, decisionLog, agentRegistry contract instances
- Export read-only provider + connected signer

FILE: lib/supabase.ts
- Supabase client with env vars

WIRE DASHBOARD:
- Pull from supabase.from('decisions') for the decisions table
- Pull from supabase.from('agent_logs') for the live agent feed
- Set up Supabase realtime subscription on agent_logs for live updates
- Total AUM: read from vault.getCurrentAllocations(), multiply by prices
- Agent status: poll every 30 seconds

WIRE VAULT CONFIG:
- Read current target allocations from vault contract
- NexaID: read nexaidHash from AgentRegistry for connected wallet
- Show "Verified" if hash is non-zero

WIRE DECISION LOG:
- Read from Supabase decisions table
- For each decision with a reasoning_hash: add "View on-chain" link
  URL: hashkey.blockscout.com/tx/{tx_hash}

WIRE SETTLEMENT:
- On submit: call POST /api/settle with form data
- API route calls settlementAgent.settleYieldPayment
- Show HSP reference on success
- Refresh settlement history table

CONNECT WALLET:
- Add MetaMask/wallet connect button
- Auto-switch to HashKey Chain testnet (chainId 133)
  window.ethereum.request({ method: 'wallet_switchEthereumChain',
    params: [{ chainId: '0x85' }] })  // 133 in hex
- If chain not added: add it automatically

END-TO-END TEST:
1. Connect wallet (shows NexaID verified)
2. Dashboard loads — portfolio chart shows real allocations
3. Agent feed shows Monitor events in real-time
4. Manually trigger drift by calling MockPriceFeed.setPrice()
5. Watch live feed: Monitor → Decision (Claude reasoning appears) → Execution (TX
link)
6. Telegram notification fires
7. Dashboard updates with new allocation

Show me terminal output — no TypeScript errors.
Show me dashboard screenshot with live data.
```

PART 8 — DEMO VIDEO SCRIPT

3 minutes maximum. Word for word. Institutional judges. Speak professionally — not like a startup pitch, like a product demo to a family office CIO.

Time	Script	Show On Screen
0:00–0:25	HashKey Chain now hosts tokenized real-world assets — silver tokens, securities, structured funds. But there are zero autonomous management tools for institutions holding these positions. APEX is that tool — four AI agents that monitor, decide, execute, and settle RWA portfolio actions with no human in the loop.	*Dashboard homepage. Portfolio allocation donut chart. 4 agent status indicators.*
0:25–0:55	The Monitor Agent reads Chainlink price feeds on HashKey Chain every 60 seconds. Right now I'll push a price change to show what happens. Silver token price just spiked 18%. Watch the Monitor Agent detect the drift.	*Terminal: Monitor Agent running. Push MockPriceFeed price update. Console shows: 'Drift detected: Silver 42.1% vs target 30%'.*
0:55–1:30	The Decision Agent calls Claude to reason about this drift. The reasoning is structured, quantified, and explainable. Watch the output. Claude recommends reducing Silver by 12%, redistributing to MMF. Confidence 91%. Reasoning stored to Supabase, keccak256 hash stored on HashKey Chain DecisionLog contract — immutable audit trail for institutional compliance.	*Dashboard Decision Log tab. Claude JSON output appears live. reasoning_hash shown. Link to blockscout.com showing on-chain log.*
1:30–2:00	The Execution Agent builds and signs the rebalancing transaction to PortfolioVault.sol on HashKey Chain testnet. Transaction confirmed in under 2 seconds. The portfolio is now back within target allocation.	*blockscout.com showing confirmed TX. Dashboard portfolio chart updates. Telegram notification pops on phone.*
2:00–2:30	The Settlement Agent handles yield distribution through HSP — HashKey Settlement Protocol. When Michael withdraws quarterly yield, instead of a bank wire, APEX creates an HSP payment request, executes the on-chain withdrawal with the HSP reference stored on-chain, and routes the funds through compliant settlement rails.	*Settlement page. Fill form: 1000 USDC to recipient. Click settle. HSP Reference ID appears. TX on blockscout with HSP ref stored.*
2:30–3:00	Three integrations that earn extra points with HashKey judges: Chainlink for real oracle data, HSP for compliant settlement, NexaID for accredited investor identity gating. APEX is institutional-grade RWA portfolio management — AI-powered, fully autonomous, built natively on HashKey Chain. We are raising to onboard the first 20 family offices in Hong Kong and Singapore.	*NexaID verified badge in vault config. Chainlink feed address shown. Close on APEX dashboard full view.*

Recording tip: Do not record as a screen recording only. Show your face for the first and last 30 seconds — judges funded by HashKey Capital are betting on a person, not a product. During the demo itself, screen only is fine.

PART 9 — SUBMISSION CHECKLIST + POST-HACK PLAN

Submission Checklist (Due April 15, 23:59 GMT+8)

> *Pre-screening is April 14 — submit at least 24 hours early. The system auto-closes at deadline. Zero exceptions.*

- Project deployed and live on HashKey Chain testnet (Chain ID: 133)
- All 4 contracts verified on hashkey.blockscout.com
- GitHub repo is PUBLIC — judges will review code
- README.md: problem, solution, tech stack, setup instructions, contract addresses
- 3-minute demo video recorded and uploaded (YouTube unlisted or Loom)
- Monitor Agent → Decision Agent → Execution Agent cycle demonstrated in video
- HSP payment reference stored on-chain and shown in video
- NexaID integration shown (even as a stub if sandbox takes time)
- Chainlink price feed integration shown (real feed or MockPriceFeed clearly labeled)
- DoraHacks submission form complete: project name, track (AI), description, team, video link, GitHub link
- Project description mentions: autonomous AI agent, HSP integration, Chainlink, NexaID

DoraHacks Submission Fields

Field	Copy-Paste Value
Project Name	APEX — Autonomous Portfolio Execution Agent
Track	AI
Tagline	4 AI agents that monitor, decide, execute, and settle RWA portfolios on HashKey Chain — no human in the loop.
Tech Stack	HashKey Chain, Solidity, Hardhat, Claude Haiku, Chainlink, HSP, NexaID, Next.js, Supabase
GitHub	https://github.com/henrysammarfo

Description (copy-paste into DoraHacks)

> *APEX is a 4-agent autonomous RWA portfolio management system built natively on HashKey Chain. The Monitor Agent reads Chainlink price feeds every 60 seconds to detect portfolio drift. The Decision Agent calls Claude Haiku for intelligent, explainable rebalancing decisions stored with keccak256 hashes on DecisionLog.sol. The Execution Agent submits*

rebalancing transactions to PortfolioVault.sol on HashKey Chain testnet. The Settlement Agent routes yield withdrawals through HSP for compliant cross-border payment settlement. NexaID gates vault access to accredited investors. APEX is the first autonomous AI portfolio management system on HashKey Chain, targeting institutional digital asset managers managing on-chain RWA positions.

Post-Hackathon Plan

Immediate — Within 48 Hours of Results

- Email every judge who interacted with APEX at the pitch — specifically the HashKey Capital representatives.
- Apply to Atlas Grant program (hashkey.xyz/grant) immediately. The hackathon judges overlap with the grant reviewers.
- Apply to HashKey Chain Eco Grants program with the APEX project as the application.

Month 1-2 — First Paying Client

- HashKey Capital's institutional network is the target. One warm introduction from a judge = one Michael-type client.
- Target: 1 family office with on-chain RWA positions, 0.10% AUM fee, $2,000/month = $2,000 ARR.
- This proves the product works on mainnet and generates the first real revenue number.

Month 3-12 — Pre-Seed

- 20 institutional clients at $2,000/month average = $480K ARR.
- Pre-seed raise: $1M from HashKey Capital + one aligned VC.
- Pitch: 'Enzyme Finance ($12M) manages manual on-chain portfolios. APEX removes the human with AI calibrated specifically for HashKey Chain's RWA ecosystem.'

The judges at this hackathon include HashKey Capital — the exact pre-seed investor APEX needs. The hackathon is not just a competition. It is a pitch to the most relevant investor in the world for this specific startup. Win the demo. Win the investment conversation.